

SEDCoT: Enhancing LLM-Based COBOL Code Translation via Symbolic Execution and Delta Debugging

PHILLIP ENTIN^{*}, Università della Svizzera italiana, Switzerland

WENCHAO GU^{*†}, Technical University of Munich, Germany

ALEXANDER KNAPP, University of Augsburg, Germany

CHUNYANG CHEN, Technical University of Munich, Germany

COBOL remains critical across banking, insurance, and government infrastructure. However, maintenance is increasingly challenging due to outdated technologies, sparse documentation, and developer retirement, necessitating code translation into modern languages like C. Traditional rule-based transcompilers yield outputs that are difficult to read and maintain, while general-purpose large language models (LLMs) achieve suboptimal correctness because COBOL is a low-resource language with distinct logic patterns. To bridge this gap, we propose SEDCoT, a novel COBOL-to-C translation framework. SEDCoT first leverages LLMs for initial translation, then combines symbolic execution with LLM guidance to generate test suites and iteratively repair semantic discrepancies. Finally, it integrates delta debugging to minimize failing tests into succinct counterexamples, accelerating automated code repair. Evaluating SEDCoT on a public COBOL-to-C dataset demonstrates that it outperforms state-of-the-art baselines by at least 12% while producing translations with substantially higher readability than rule-based alternatives.

CCS Concepts: • **Software and its engineering** → **Software maintenance tools**; **Source code generation**; • **Computing methodologies** → **Machine translation**.

Additional Key Words and Phrases: Code Translation, Code Repair, Large Language Model, Symbolic Execution, Delta Debugging

ACM Reference Format:

Phillip Entin, Wenchao Gu, Alexander Knapp, and Chunyang Chen. 2026. SEDCoT: Enhancing LLM-Based COBOL Code Translation via Symbolic Execution and Delta Debugging. *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA168 (October 2026), 24 pages. <https://doi.org/10.1145/3832259>

1 Introduction

Legacy systems remain critical across many sectors, including banking, insurance, and government infrastructure [56]. Despite their age, these systems continue to support mission-critical operations — processing \$3 trillion in commerce transactions every day [8]. However, maintaining them is increasingly difficult due to outdated technologies, sparse digital documentation, and the retirement of original developers [8, 56]. As technical debt accumulates, organizations face growing pressure to modernize their infrastructure to improve maintainability, scalability, and compliance [8, 56].

^{*}Both authors contributed equally to this work.

[†]Corresponding author.

Authors' Contact Information: [Phillip Entin, phillip.entin@usi.ch](mailto:phillip.entin@usi.ch), Università della Svizzera italiana, REVEAL @ Software Institute, Lugano, Switzerland; [Wenchao Gu, wenchao.gu@tum.de](mailto:wenchao.gu@tum.de), Technical University of Munich, Munich, Germany; [Alexander Knapp, alexander.knapp@uni-a.de](mailto:alexander.knapp@uni-a.de), University of Augsburg, Augsburg, Germany; [Chunyang Chen, chun-yang.chen@tum.de](mailto:chun-yang.chen@tum.de), Technical University of Munich, Munich, Germany.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/10-ARTISSTA168

<https://doi.org/10.1145/3832259>

Among legacy technologies, COBOL stands out as particularly entrenched and challenging. Originally developed in the 1960s for business applications [8, 56], COBOL remains widely used today; estimates suggest that more than 220 billion lines of COBOL code are still operational [51]. These programs are deeply embedded in financial systems, mainframes, and government platforms [56].

COBOL's verbose, data-centric syntax and reliance on legacy constructs make it increasingly incompatible with contemporary development practices. Translating COBOL into mainstream programming languages therefore requires more than a purely syntactic mapping. COBOL programs commonly employ hierarchical data definitions, atypical control-flow constructs, and tight coupling between data layout and program logic [22]; these characteristics create a substantial semantic gap between COBOL and mainstream programming languages.

Rule-based tools such as the GnuCOBOL compiler can emit code in mainstream programming languages from COBOL source [36]. However, because of the significant grammatical and idiomatic differences between COBOL and contemporary languages, the generated code is often difficult for human developers to read and maintain, and typically requires extensive manual refactoring before it is suitable for long-term maintenance. Correctness would indeed be paramount if the translation were like a compilation—"compile, run, and forget." But readability will outweigh "simple" correctness if the translated code has to be worked with in any respect, in particular, if it will have to be maintained as is the usual case in replacing legacy software.

Driven by this critical need for readability, recent advances in large language models (LLMs) have produced impressive results in program translation [17, 21, 24, 43, 62]. Unlike rule-based methods, LLM-generated translation code exhibits readability comparable to human-written programs with more natural coding styles and idiomatic expressions. Nevertheless, while LLMs excel in readability, they suffer from a sharp accuracy drop compared with rule-based approaches, and this limitation mainly stems from two key factors. First, COBOL is a low-resource language: far less open-source COBOL code is available for training compared with mainstream programming languages, which limits model exposure to COBOL-specific idioms. Second, COBOL's grammar and design contain features that do not map cleanly to mainstream programming languages — for example, COBOL's support for decimal/fixed-point arithmetic and implicit variable initialization at program start [10, 56] — making it hard for models to learn correct cross-language behaviors from data in other languages. These factors help explain why prior LLM-based COBOL translation efforts [17] lag behind LLM performance on translations between mainstream programming languages.

To address the severe accuracy degradation of LLM-based COBOL translation while preserving readability, we propose SEDCoT, a novel framework operating in three phases: initial translation, test-case generation, and code repair. First, we prompt LLMs to produce candidate C translations from COBOL. Second, we combine symbolic execution with LLM guidance for test generation: the symbolic execution component systematically explores branches and synthesizes inputs to maximize coverage and expose semantic mismatches [7], while LLMs increase input diversity. Third, we execute the generated test suite against candidates and supply failing tests to the LLM for automated repair. For complex failures, we apply delta debugging to minimize failing inputs into succinct counterexamples, effectively reducing the search space and helping the LLM localize bugs [65].

We evaluate SEDCoT on 319 COBOL programs from IBM's CodeNet dataset, translating them into C due to its prevalence in legacy modernization pipelines. For each program, we synthesize approximately 500 diverse test cases by varying input values, mixing data types, altering whitespace, and injecting non-printable characters. Experimental results show that SEDCoT substantially improves translation accuracy by at least 12% over state-of-the-art baselines, demonstrating more robust and semantically faithful correctness.

This work makes the following key contributions:

- We propose a novel LLM-based code translation framework that integrates initial code translation with translated code repair, thereby improving overall translation accuracy.
- We propose a symbolic-execution-based approach to automatically generate test cases with higher line and branch coverage, which are more effective in triggering potential bugs in the initially translated code and guiding the LLM during the repair process.
- We propose a delta-debugging-based method to simplify complex test cases which are hard for LLMs to repair, enabling LLMs to more effectively localize buggy code segments.
- We conduct a comprehensive evaluation on a public dataset against two state-of-the-art (SOTA) baselines. Experimental results demonstrate that our method significantly outperforms existing SOTA approaches.

2 Background

2.1 Symbolic Execution

Symbolic execution utilizes constraint solvers to derive inputs that explore distinct program paths [7]. UTBot industrializes this, employing a hybrid concolic engine combining an enhanced KLEE symbolic virtual machine with smart fuzzing to maximize code coverage while mitigating path explosion [57]. Notably, UTBot synthesizes raw KLEE-derived inputs into readable, structured regression test suites based on the Google Test framework, featuring automated method variable generation, assertions, and mock stubs.

2.2 Delta Debugging

When a program exhibits erroneous behavior on a particular input, the input often contains superfluous elements that are not necessary to trigger the fault. Delta debugging, first introduced by Zeller [65], is a widely used technique to automatically minimize such failure-inducing inputs. The core idea is to iteratively partition the input and test whether individual subsets are still capable of reproducing the failure. By systematically discarding irrelevant components, delta debugging converges toward a minimal configuration that preserves the fault-inducing property.

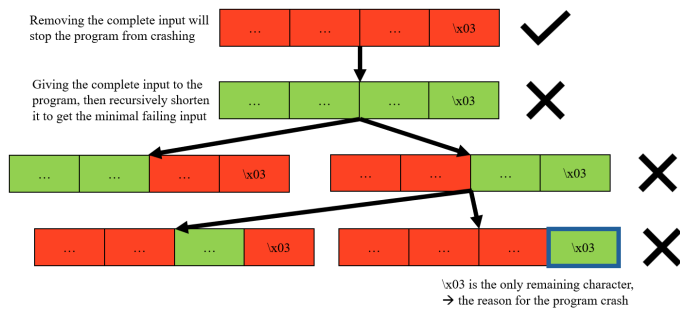


Fig. 1. Example of delta debugging: a string containing `\x03` is reduced to the minimal failing input `\x03`.

For example, suppose a program fails when given a string containing `"\x03"`. Delta debugging may reduce this input to `"\x03"` if the shorter prefix alone suffices to reproduce the bug, as illustrated in Figure 1. In this case, the non-printable character `"\x03"` is likely the fault trigger, potentially due to improper handling of non-printable characters in the input parser. Such test case reduction is not only valuable for human developers—by simplifying the debugging process—but also plays a

key role in our work: it enables large language models (LLMs) to focus on the essential features of failing cases rather than being distracted by irrelevant context.

2.3 COBOL Translation

Open-source solutions like GnuCOBOL enable the compilation and execution of COBOL today [36] by translating COBOL code into an intermediate C representation [19]. GnuCOBOL successfully passes 9700 out of 9748 tests in the NIST COBOL 85 test suite [19], a benchmark verifying compliance with the COBOL 85 standard [35]. While this high coverage demonstrates that GnuCOBOL preserves the semantic correctness of the original programs, its rule-based translation mechanism yields C code that is often difficult for humans to read or maintain.

For example, a simple COBOL program checking for consecutive doublets is translated into 283 lines of C code, supplemented by two headers of 26 and 59 lines, whereas a human-written C solution requires only about 20 lines. Consequently, we adopt GnuCOBOL as our primary rule-based baseline to compare its readability against the code produced by SEDCoT.

3 Methodology

3.1 Overview

To improve COBOL translation accuracy, we propose a novel approach named SEDCoT, which integrates symbolic execution and delta debugging into test case generation and refinement for code repair during the translation. As illustrated in Figure 2, SEDCoT is organized into three sequential phases: initial code translation, test case generation, and code repair. Notably, symbolic execution is exclusively adopted in the second phase to produce test cases that are subsequently utilized in the third phase, and these two phases run strictly in sequence without forming any iterative loops.

In the initial translation phase, an LLM translates COBOL into the target language to produce candidate code. During test case generation, we construct verification inputs comprising two types: rule-based test cases generated via rule-guided COBOL translation combined with symbolic execution for high code coverage, and LLM-generated test cases. Finally, in the code repair phase, these test cases are executed on the initial translation, and execution feedback is routed to the LLM for iterative error correction. If repeated repair attempts fail, delta debugging simplifies the failing test cases to reduce failure analysis difficulty. This simplified information is then fed back to the LLM to guide subsequent repair and yield the finalized target-language code.

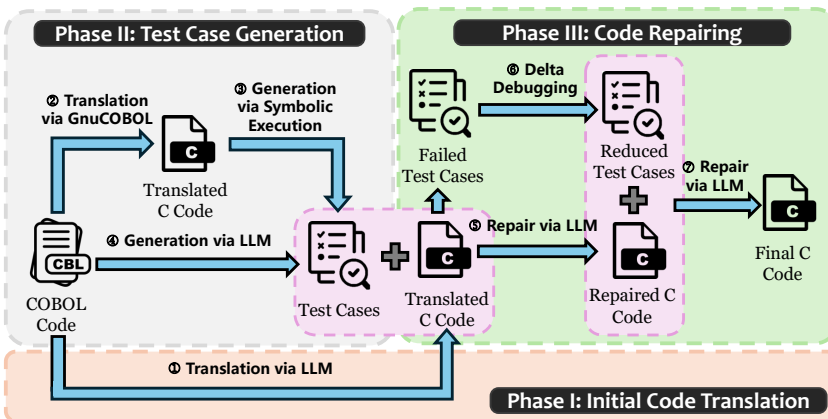


Fig. 2. Overview of our test-driven COBOL-to-C translation refinement pipeline.

3.2 Initial Code Translation

3.2.1 Translation via LLM. At the first stage, we leverage LLMs to translate the target COBOL code into the code with target programming language, which serves as the initial translation template for subsequent refinement. The translation process can be formally expressed as:

$$P(C \mid I) = \prod_{t=1}^T P(c_t \mid c_{<t}, I), \quad (1)$$

where I denotes the input COBOL code, C represents the translated code, T is the sequence length, and $P(\cdot)$ corresponds to the conditional probability distribution over tokens.

Since the initially translated code may contain compilation errors, we attempt to compile the generated code. Let $C(\cdot)$ denote the compilation function that maps a candidate program C to a diagnostic outcome E :

$$E = C(C), \quad (2)$$

where $E = \emptyset$ indicates successful compilation, and $E \neq \emptyset$ corresponds to a set of compilation errors.

If $E \neq \emptyset$, we provide both the erroneous code C and the compiler feedback E to the LLM, enabling iterative refinement. Formally, the refinement step at iteration k can be represented as:

$$C^{(k+1)} = \text{LLM}(I, C^{(k)}, E^{(k)}), \quad (3)$$

where $C^{(k)}$ is the candidate code at iteration k , and $E^{(k)}$ is the corresponding compilation feedback. After K iterations, the final code C^* is selected as the candidate with the minimal number of compilation errors. Note that the repair process requires executable code; therefore, the subsequent repair procedure will be skipped if C^* fails to compile.

3.3 Test Case Generation

3.3.1 Translation via GnuCOBOL & Generation via Symbolic Execution. To validate the correctness of the translated code and to provide feedback that can assist LLMs in refining the translation, we generate test cases for the original COBOL code. Let $\mathcal{T}(I)$ denote the set of test cases generated for the COBOL program I . These test cases are subsequently used for evaluating the translated code C .

Most prior approaches rely on LLMs to automatically generate test cases. However, such test cases often suffer from insufficient code coverage, resulting in weak test sets that are easily passed and thus less effective. To address this limitation, we employ symbolic execution techniques, which are capable of systematically exploring program paths to achieve higher code coverage. Formally, let $X = \{x_1, x_2, \dots, x_n\}$ represent the set of program inputs, which are treated as symbolic variables. The symbolic execution engine explores feasible execution paths $\mathcal{P} = \{p_1, p_2, \dots, p_m\}$, producing a set of test cases T corresponding to each path:

$$T = \{t_i \mid t_i \text{ satisfies path } p_i \in \mathcal{P}\}. \quad (4)$$

To the best of our knowledge, no open-source symbolic execution engine currently supports direct test case generation for COBOL. To leverage the proven effectiveness of symbolic execution-based test cases [27], we adopt the following workaround: the rule-based compiler translates COBOL programs into functionally equivalent intermediate code in the target programming language. This enables the use of existing symbolic execution engines to generate test cases that capture the behavioral aspects of the original COBOL program.

Formally, let I_C denote the intermediate representation of the COBOL program. The symbolic execution engine $\mathcal{S}$ is then applied to I_C to generate the test cases:

$$T = \mathcal{S}(I_C, X), \quad (5)$$

where X denotes the set of symbolic input variables. Each generated test case $t_i \in T$ corresponds to a concrete assignment of X that triggers a unique execution path in I_C , thereby providing comprehensive coverage for subsequent evaluation of the translated code.

3.3.2 Generation via LLM. Nevertheless, symbolic execution alone may not always generate feasible test cases, as it can struggle with certain program constructs. For instance, loops and nested branches may cause path explosion, while conditions involving non-linear arithmetic can result in constraints that are difficult for SMT solvers to resolve. As a consequence, some execution paths may remain uncovered, leaving no concrete test cases available. To mitigate this limitation, we complement symbolic execution with LLM-generated test cases. Let $\mathcal{T}_{\text{LLM}}(I)$ denote the set of test cases generated by the LLM from the COBOL source code I (code translated by GnuCOBOL is lengthy and obscure, making it difficult for LLMs to comprehend and generate test suites), following a procedure similar to the UniTrans approach [62]:

$$\mathcal{T}_{\text{LLM}}(I) = \text{LLM}_{\text{GenerateTests}}(I). \quad (6)$$

The final set of test cases for evaluating the translated C code is then obtained by combining the symbolic execution and LLM-generated test cases:

$$T_{\text{final}} = T \cup \mathcal{T}_{\text{LLM}}(I), \quad (7)$$

where T is the set of symbolic execution-based test cases defined previously.

Finally, the COBOL program is executed with the inputs in T_{final} , and the corresponding outputs are recorded as the ground truth O_{COBOL} :

$$O_{\text{COBOL}} = \{o_i \mid o_i = \text{Execute}(I, t_i), t_i \in T_{\text{final}}\}. \quad (8)$$

These ground-truth outputs are used to validate the correctness of the translated code and to provide feedback for subsequent refinement.

3.4 Code Repairing

3.4.1 Repair via LLM. Once the test cases T_{final} and a compilable code C are obtained, we first evaluate the generated code against the provided test cases. Let the execution of candidate code $C^{(k)}$ on test case t_i produce output $o_i^{(k)}$, and define the set of failing test cases as

$$\mathcal{E}^{(k)} = \{t_i \in T_{\text{final}} \mid o_i^{(k)} \neq o_i\}, \quad (9)$$

where o_i is the corresponding ground-truth output from the COBOL program.

The repair pipeline is initiated to iteratively re-prompt the LLM to correct failing test cases and produce updated translations. At iteration k , the LLM receives as input the original COBOL code I , the current candidate $C^{(k)}$, and the execution results for only the failing test cases $(t_i, o_i^{(k)}, o_i)_{t_i \in \mathcal{E}^{(k)}}$, following the approach of [43]:

$$C^{(k+1)} = \text{LLM}_{\text{Repair}}\left(I, C^{(k)}, (t_i, o_i^{(k)}, o_i)_{t_i \in \mathcal{E}^{(k)}}\right). \quad (10)$$

During the repair phase, we additionally verify the output format to ensure that superficially correct values with extraneous spaces, leading zeros, or numeric formatting differences are treated as failures.

A maximum number of repair iterations K_{max} is enforced. After K_{max} attempts, the final candidate C^* is selected as the version that passes the largest number of test cases. Let

$$F^{(k)} = \sum_{t_i \in T_{\text{final}}} \mathbf{1}[o_i^{(k)} = o_i] \quad (11)$$

denote the total number of passing test cases. Then

$$C^* = \arg \max_{0 \leq k \leq K_{\max}} F^{(k)}. \quad (12)$$

If a candidate fails to compile during the repair process, the most recent successfully compiled version is restored and used in subsequent iterations. This procedure ensures that the selected final version is both compilable and achieves the highest overall test-case success.

3.4.2 Delta Debugging. When a candidate translation continues to fail certain test cases after the compilation and repair stages, naively re-prompting the LLM often yields diminishing returns, as the remaining failures involve errors that are difficult for the model to identify or reason about. Building on the observation of Yang et al. [62] that iterative repairs tend to saturate quickly, we adopt delta debugging to isolate the root causes of these residual errors and provide them to the LLM as explicit repair guidelines.

Let $\mathcal{E}^*$ denote the set of failing test cases remaining after standard repair iterations. For each failing input $t_i \in \mathcal{E}^*$, delta debugging produces a minimal counterexample $t_i^{\min}$ that still reproduces the error:

$$t_i^{\min} = \text{DeltaDebug}(t_i), t_i \in \mathcal{E}^*, \quad (13)$$

where $\text{DeltaDebug}(\cdot)$ iteratively shortens and simplifies the input until a minimal reproducible input is obtained [65]. The resulting set of minimal counterexamples is

$$\mathcal{E}^{\min} = \{t_i^{\min} \mid t_i \in \mathcal{E}^*\}. \quad (14)$$

```

1 DATA DIVISION.
2 ...
3 01 S PIC X(01).
4 PROCEDURE DIVISION.
5     ACCEPT S
6     ACCEPT T
7     IF S = T THEN DISPLAY "foo" END-IF
8     STOP RUN.
```

Listing 1. Minimal COBOL example

These minimal inputs often reveal the underlying fault. When $t_i^{\min}$ reduces to an empty string or a single character, the failure is typically caused by COBOL's implicit initialization semantics (e.g., variables in working storage are automatically initialized in GnuCOBOL [10]). Listing 1 illustrates a minimal example of this issue: the program requests two string inputs from the user and prints foo if they are equal. However, if the user provides no input, the program still prints foo due to GnuCOBOL's automatic variable initialization. Longer minimal inputs containing unusual whitespace or control characters typically indicate unsafe input handling or incorrect assumptions regarding input length.

To avoid overwhelming the LLM with redundant examples, we normalize and deduplicate failing test cases. Let $\text{Canonical}(t_i^{\min})$ denote the normalized form of $t_i^{\min}$ (e.g., replacing ASCII codes below 32 with a dot). The final set of unique minimal counterexamples is

$$\mathcal{E}^{\text{canon}} = \{\text{Canonical}(t_i^{\min}) \mid t_i^{\min} \in \mathcal{E}^{\min} \text{ and duplicates removed}\}. \quad (15)$$

By providing only the unique, minimal failing inputs, this delta-debugging reduction step supplies the LLM with precise, high-level guidance, rather than a large number of opaque examples. Consequently, we achieve better repair outcomes in the final stage without extra LLM calls. Delta

debugging helps LLMs precisely identify errors and finish fixes efficiently, avoiding repeated model invocations for iterative reasoning and trial repairs.

However, an early application of delta debugging may prematurely constrain the search space of LLMs and bias the subsequent repair steps toward overly specific fixes. Suppose that delta debugging adopts a test oracle that only discriminates between *fail* and *pass* outcomes. In this scenario, delta debugging performs input minimization merely based on the preservation of failure behavior, with no consideration for the distinct underlying causes of failures. A representative example is given by the pseudocode in Listing 2:

```

1 if input == "abc": crash
2 elif input == "ab": crash
3 elif input == "a": return expected

```

Listing 2. Minimal DD Example

When the input "abc" is given, the program crashes. Since the input "ab" also triggers a crash, delta debugging reduces the original input to "ab" as the minimal failure-inducing input. As a result, the bug in the branch corresponding to "abc" becomes unobservable.

To address this issue, SEDCoT first attempts to fix the bug using the full input in the initial several rounds, and only employs delta debugging in the final round to perform input minimization.

3.4.3 Repair via LLM. Once the reduced test cases $\mathcal{E}^{\text{canon}}$ are generated, their corresponding execution results o_i^{min} , together with the current translated code C , are used to guide the final repair step:

$$C^* = \text{LLM}_{\text{FinalRepair}}\left(C, (t_i^{\text{min}}, o_i^{\text{min}}, o_i) \mid t_i^{\text{min}} \in \mathcal{E}^{\text{canon}}\right), \quad (16)$$

where o_i denotes the ground-truth output produced by the original COBOL program. This formulation ensures that the LLM focuses specifically on the remaining minimal failing inputs, leveraging precise execution feedback to generate the final repaired code C^* . In addition, we design instructions that explicitly encourage the LLM to improve robustness in input handling and variable initialization—issues that frequently arise in COBOL translation.

4 Experimental Settings

In this section, we introduce the datasets, baselines and LLMs being evaluated in the experiments.

4.1 Dataset

Table 1. Statistics for Lines of Code

	Min	Median	Mean	Max
LOC	9.0	32.0	37.2	210.0

Table 2. Percentage Distribution of different Lines

	<20	20 ~ 50	51 ~ 100	>100
Num	58 (18.2%)	198(62.1%)	54(16.9%)	9 (2.8%)

Building upon the work of Gandhi et al. [17], we adopt the IBM CodeNet dataset and extract 322 COBOL programs that not only compile successfully but also pass their associated sample tests (marked as "accepted") for our evaluation. However, the IBM dataset does not provide dialect version information, which leads to compilation and reproducibility issues. We therefore require all programs to be reproducibly compilable using a controlled, uniform toolchain (GnuCOBOL); under this stricter and experimentally necessary criterion, 319 programs are retained. GnuCOBOL is a reliable open-source COBOL-to-C translation tool to convert COBOL programs into C code. We selected GnuCOBOL over alternative dialects for two primary reasons. First, Project CodeNet

metadata explicitly designates "OpenCOBOL 1.1.0"—the direct predecessor of GnuCOBOL—as the source environment for all submissions. Second, migrating this legacy source code to alternative dialects would invariably introduce severe compilation discrepancies. As documented in its official manual [19], GnuCOBOL achieves a 98.3% pass rate on the NIST COBOL85 test suite, with only the "Advanced facility" subsets of the "CM - COMMUNICATION SECTION tests", "DB - Debugging facilities tests", and "OB - Obsolete facilities tests" remaining untested. Despite not achieving 100% accuracy, its output is still regarded as the ground-truth translated code for this study. Table 1 and Table 2 show the statistical results about the dataset. For system execution tools, KLEE [7] is adopted for rule-based test case generation.

We attempted to validate our method against other mainstream languages. However, our framework requires reliable baseline transcompilers to support subsequent test generation and behavior comparison. Currently, robust open-source translation tools for other target languages remain unavailable: for instance, translating COBOL to Java via an open-source tool [53] yielded only 11 successful translations, an insufficient sample size for meaningful statistical evaluation. Consequently, we limit our target language to C for all experiments in this study.

4.2 Evaluation

Because the dataset's original test cases are overly simplistic, we adopt a differential testing paradigm using a held-out golden test suite. We set the generated input volume to 500 per program to balance computational overhead and anomaly detection. Mutating from each benchmark's original input, we generate these 500 test cases evenly across five strategies (100 samples each): type-preserving shuffling, mixed-type shuffling, whitespace perturbation, variable-length truncation with non-printable characters, and variable-length truncation with mixed character replacement. This test suite achieves 88.42% statement coverage and 52.97% branch coverage. Crucially, no translated program passes the entire suite, demonstrating its comprehensiveness in uncovering behavioral discrepancies.

Since native COBOL code cannot be directly executed on modern hardware, we leverage GnuCOBOL to compile and run the original COBOL programs, and take its execution behavior as the ground-truth oracle. We then compare the runtime outputs of LLM-translated programs against this GnuCOBOL-based reference to rigorously examine the behavioral equivalence and robustness of translation results. In our differential testing pipeline, GnuCOBOL therefore acts as the authoritative execution standard. Finally, we quantify the performance of all methods by computing the average test case pass ratio over the 500 perturbed inputs per program.

4.3 Baselines and LLMs evaluated

We evaluate SEDCoT on four representative large language models that vary in architecture, parameter scale, and context-window size. This selection enables a systematic assessment of SEDCoT's performance across different generative systems. All LLMs were deployed and executed on cloud platforms. Specifically, GPT-4o-mini ran on OpenAI's service infrastructure, and other models were accessed via diverse providers supported by OpenRouter [39].

- *Gemma-3-27b-it* (Google) [42]: an instruction-tuned model with 27 billion parameters and a context window of 128,000 tokens.
- *Llama-3.3-70b-instruct* (Meta) [40]: an instruction-tuned model with 70 billion parameters and a context window of 131,000 tokens.
- *Qwen-2.5-coder-32b-instruct* (Qwen) [41]: a coder-oriented model with 32 billion parameters and a context window of 32,000 tokens.

- *GPT-4o-mini-2024-07-18* (OpenAI) [38]: a general-purpose model with undisclosed parameter count; its context window is accessed via the official OpenAI API.

In addition, we evaluate the efficacy of SEDCoT by comparing it against one rule-based approach and two state-of-the-art LLM-based baseline methods:

- **TinyCOBOL**: an early open-source COBOL 85 compiler that directly translates COBOL source code into x86 assembly language.
- **UniTrans** [62]: an iterative repair procedure driven solely by LLM-generated tests, originally designed for program translation tasks in modern programming languages such as Java, Python and C++. As a representative general LLM-based code translation framework, it is adopted to verify the applicability of such mainstream solutions to legacy code translation scenarios. We retain its full pipeline and only adjust prompts and compiler settings to support COBOL-to-C translation. For fair comparison, we limit its iterative repair to three rounds, consistent with SEDCoT.
- **High-Resource Java Refinement (HRJR)** [17]: a three-step logic-readability-logic refinement sequence proposed for COBOL-to-Java translation. We retained the complete workflow of HRJR, and only adjusted the LLM prompt to generate C code instead of Java, while replacing the compiler accordingly.

4.4 Implementation Details

In our experiments, the temperature of all LLMs was set to zero to eliminate sampling variance [50]. To mitigate the high time complexity of symbolic execution, we imposed maximum allowances for each COBOL program. Specifically, KLEE was configured to explore up to 80 million expressions, restricted to an overall time limit of 20 minutes, 2 minutes per state, and 5 minutes for coverage measurement. Test generation via KLEE was conducted on an Intel i5-11300H laptop (16GB RAM) over 12 hours, successfully producing test suites for 292 programs with an average runtime of 2.5 minutes. All prompts used in our method are provided in the replication package's Appendix.

Table 3. Statistics of Test Cases Generated by Symbolic Execution and LLMs.

	Symbolic	Gemma-3	GPT-4o-mini	Llama-3.3	Qwen-2.5-coder
Mean	15.25	15.82	14.06	7.22	35.32
Median	17.00	13.00	8.00	7.00	10.00
Min	1.00	1.00	2.00	2.00	2.00
Max	144.00	28.00	1001.00	30.00	1001.00
Succeed Cases	292	293	308	303	303

Table 4. Overlap of Test Cases Generated by Different Approaches.

	Sym&Gem	Sym&GPT	Sym&Lla	Sym&Qwe	Gem&GPT	Gem&Lla	Gem&Qwe	GPT&Lla	GPT&Qwe	Lla&Qwe
Overlap	270 (84.6%)	282 (88.4%)	277 (86.8%)	276 (86.5%)	284 (89.0%)	279 (87.5%)	280 (87.8%)	294 (92.2%)	293 (91.8%)	290 (90.9%)

Table 3 summarizes the statistics of the test cases generated by both symbolic execution and LLMs. Here, “Succeed Cases” refers to the number of COBOL programs for which test cases were successfully generated. Since program complexity varies significantly, the number of test cases required to achieve high line and branch coverage also differs widely. Therefore, we did not impose an upper bound on the number of test cases, allowing both symbolic execution and LLM-based

methods to generate cases until they naturally terminated. Table 4 summarizes the number of overlapping test cases generated by different approaches. No test cases are uniquely generated by either one LLM alone or symbolic execution. In addition, all experimental results are reported using the 319 test cases. The repair stage is skipped if no test cases are provided for repair, yet the translated code is still evaluated against the golden test suite.

For experimental consistency, we strictly follow the original settings of each baseline: HRJR adopts the native test cases from CodeNet for program repair; UniTrans leverages LLM-generated test cases to conduct repair as originally designed. In terms of our proposed SEDCoT, we employ both LLM-generated and symbolic execution-derived test cases for program repair in RQ1. By contrast, selective combinations of these two types of test cases are adopted in RQ2 and RQ4 to satisfy the requirements of ablation studies.

In general, a large number of repair retries may lead to diminishing performance [62]. For example, Gandhi et al. [17] limit the maximum number of repair iterations to three in their COBOL translation approach. To ensure consistency with prior work and enable a fair performance comparison, we similarly set the total number of repair attempts per test suite to three. Concretely, in the first code repair stage (Step 5 of Phase 3), LLMs were allowed up to two retries. In the second code repair stage (Step 7), the maximum number of retries was set to one; however, in the event of compilation failures, we allowed up to two additional attempts solely to address compilation errors.

5 Evaluation

We aim to answer the following research questions (RQs):

- **RQ1:** How effective is SEDCoT in translating COBOL programs compared with SOTA baseline methods?
- **RQ2:** To what extent do the individual components of SEDCoT to the overall translation quality?
- **RQ3:** Can symbolic execution test cases reveal more latent bugs than LLM test cases?
- **RQ4:** How does the number of repair attempts affect the performance of SEDCoT?
- **RQ5:** How does the readability of code produced by SEDCoT compare with that of rule-based translation approaches?

5.1 RQ1: How effective is SEDCoT in translating COBOL programs compared with SOTA baseline methods?

Table 5. Performance comparison of different approaches with various LLMs (best performance in bold).

	Gemma-3	GPT-4o-mini	Llama-3.3	Qwen-2.5-coder
Vanilla LLM	0.287	0.393	0.338	0.345
HRJR	0.455	0.380	0.502	0.504
UniTrans	0.516	0.440	0.514	0.531
SEDCoT	0.579 (↑ 12.2%)	0.718 (↑ 63.2%)	0.607 (↑ 18.1%)	0.668 (↑ 25.8%)
TinyCOBOL	0.207	0.207	0.207	0.207
GnuCOBOL	1.000	1.000	1.000	1.000

Table 5 compares the overall performance of SEDCoT with the baselines across different LLMs. Table 6 presents detailed status statistics of translated codes across different stages, including uncompileable programs, compileable codes failing generated test suites, and successfully repaired codes. Notably, no programs can fully pass the golden test suites. First, we find that simply adopting

Table 6. Status of Translated Codes at Different Phases

		Gemma-3	GPT-4o-mini	Llama-3.3	Qwen-2.5-coder
Compile-error	Vanilla LLM	1.3% (4/319)	7.2% (23/319)	12.2% (39/319)	14.4% (46/319)
	HRJR	4.7% (15/319)	15.4% (49/319)	16.6% (53/319)	17.2% (55/319)
	UniTrans	1.3% (4/319)	7.2% (23/319)	12.2% (39/319)	14.4% (46/319)
	SEDCoT	2.8% (9/319)	0.9% (3/319)	12.9% (41/319)	6.0% (19/319)
Failed	Vanilla LLM	91.1% (275/302)	87.4% (270/309)	88.6% (242/273)	85.8% (254/296)
	HRJR	59.2% (180/304)	25.2% (68/270)	65.8% (175/266)	85.6% (226/264)
	UniTrans	83.8% (264/315)	19.6% (58/296)	82.5% (231/280)	83.2% (227/273)
	SEDCoT	81.6% (253/310)	77.8% (246/316)	73.0% (203/278)	74.7% (224/300)
Repaired	HRJR	2.2% (4/180)	0.0% (0/68)	1.5% (4/175)	1.9% (5/226)
	UniTrans	1.9% (5/264)	1.7% (1/58)	2.6% (6/231)	2.6% (6/227)
	SEDCoT	12.6% (32/253)	26.4% (65/246)	27.6% (56/203)	25.9% (58/224)

LLMs for COBOL code translation yields low accuracy, below 40%. In contrast, our proposed approach significantly improves accuracy and outperforms state-of-the-art baselines. Specifically, SEDCoT achieves at least a 12.2% improvement and approximately 30% average improvement across all LLMs. Furthermore, the relative performance improvement of SEDCoT depends on the base LLM's translation capability: the stronger the base LLM, the larger the potential improvement provided by SEDCoT. For instance, Gemma-3, which exhibits the lowest baseline performance, gains around 12% improvement with SEDCoT, whereas GPT-4o-mini, the best-performing LLM, sees a relative improvement exceeding 60%. Given that GnuCOBOL serves as the ground-truth oracle in our experimental setup, its performance is defined as the baseline (100%) for behavioral consistency. Notably, TinyCOBOL exhibits unsatisfactory performance due to severe compatibility limitations. It generates 32-bit assembly code that demands a dedicated 32-bit runtime environment. Although we performed syntactic adaptation to align the assembly output with our GCC compilation configuration and boosted the initial compilation success rate, 206 out of 319 programs still failed to compile owing to syntax discrepancies. As TinyCOBOL stopped receiving updates in January 2011, such compilation failures stem primarily from its inability to support modern COBOL dialects and language variants contained in the CodeNet dataset.

Interestingly, using LLMs to refactor GnuCOBOL-translated code (denoted as LLM Refactor) yields significantly lower accuracy than direct LLM translation. This occurs because rule-based translated code differs substantially from human-written programs in structure and readability. Lacking conventional coding styles and logical clarity, such translated code is not only opaque to human developers but also prevents LLMs from performing effective refactoring.

Summary 1: Although LLMs excel at generating syntactically correct, compilation-error-free code, functional correctness is not guaranteed. While SEDCoT significantly boosts LLM translation performance, this improvement is more pronounced when applied to models with stronger baseline translation capabilities.

5.2 RQ2: To what extent do the individual components of SEDCoT contribute to the overall translation quality?

To evaluate the contribution of each component to the overall performance, we conduct a comprehensive ablation study by selectively removing individual components and examining the resulting performance changes. Specifically, we construct three variants to evaluate individual component

contributions: $\text{SEDCoT}_{\text{LLMTest}}$, $\text{SEDCoT}_{\text{SymTest}}$, and $\text{SEDCoT}_{\text{w/oDelta}}$. $\text{SEDCoT}_{\text{LLMTest}}$ isolates the impact of LLM-generated test cases during the repair stage by removing the symbolic execution component from Phase II. $\text{SEDCoT}_{\text{SymTest}}$ exclusively utilizes test cases produced by symbolic execution tools, omitting the LLM-based test generation in Phase II. $\text{SEDCoT}_{\text{w/oDelta}}$ disables the delta debugging process in Phase III, meaning that failed test cases are fed directly into the final repair round without any simplification. Furthermore, since delta debugging inherently requires a foundational test suite to operate, and isolating either pure symbolic-based or pure LLM-based testing has already been extensively explored in literature, we omit further combinations of these two-component ablations.

Table 7. Contribution of Different Components in SEDCoT.

	Gemma-3	GPT-4o-mini	Llama-3.3	Qwen-2.5-coder
$\text{SEDCoT}_{\text{LLMTest}}$	0.488	0.547	0.513	0.503
$\text{SEDCoT}_{\text{SymTest}}$	0.601 ($\uparrow$ 3.8%)	0.663	0.594	0.630
$\text{SEDCoT}_{\text{w/oDelta}}$	0.475	0.586	0.520	0.568
SEDCoT	0.579	0.718 ($\uparrow$ 8.3%)	0.607 ($\uparrow$ 2.2%)	0.668 ($\uparrow$ 6.0%)

Table 7 presents the performance of these variants. First, we observe that SEDCoT consistently achieves the best performance across most LLMs, demonstrating the effectiveness of combining test cases generated by both symbolic execution tools and LLMs, along with delta debugging. Furthermore, by comparing $\text{SEDCoT}_{\text{LLMTest}}$ and $\text{SEDCoT}_{\text{SymTest}}$, we find that the variant using symbolic execution-generated test cases significantly outperforms the one relying solely on LLM-generated test cases. This is because symbolic execution can produce test cases covering corner cases, which are more likely to expose bugs and thus provide stronger guidance for LLMs to identify and repair faulty code. For Gemma-3, symbolic-execution-generated test cases alone yield better performance than combined test sets, further validating this conclusion. Finally, comparing $\text{SEDCoT}_{\text{w/oDelta}}$ with SEDCoT highlights the importance of delta debugging: simplifying complex failing test cases that could not be repaired in earlier rounds makes them easier for LLMs to interpret, thereby increasing the likelihood of successful repair in the final round.

Summary 2: Combining the test cases generated by symbolic execution tools and LLMs, and applying delta debugging to simplify those test cases that are difficult to repair, can significantly improve overall performance. In particular, the use of test cases generated by symbolic execution tools, together with delta debugging, makes a substantial contribution to the effectiveness of our approach.

5.3 RQ3: Can symbolic execution test cases reveal more latent bugs than LLM test cases?

Table 8. Statistics of code coverage.

Avg. coverage	Symbolic	Gemma-3	GPT-4o-mini	Llama-3.3	Qwen-2.5-coder
Line	0.852	0.884	0.879	0.883	0.876
Branch	0.469	0.524	0.515	0.525	0.522

To better understand if test cases generated by symbolic execution tools can more effectively assist LLMs in repairing translated C code, we first compare the code coverage achieved by test

cases from both symbolic execution tools and LLMs, as shown in Table 8. Coverage was calculated based on the number of programs for which test cases can be successfully generated for each model. For example, ChatGPT’s average coverage was computed over 308 programs, Gemma’s over 293 programs, and so forth. Interestingly, the results show no significant difference in coverage between the two approaches. In some cases, LLM-generated test cases even achieve slightly higher line and branch coverage than those produced by symbolic execution.

At first glance, this observation seems contradictory. However, the key lies in the limitations of traditional coverage metrics such as line and branch coverage. These metrics are inherently syntactic: they only measure whether certain statements or branches have been executed, without capturing how test inputs interact with program semantics or whether they are capable of exposing faulty behavior. Symbolic execution, in contrast, tends to generate inputs that exercise edge cases—for example, non-printable characters, malformed data, or extreme boundary values. Such inputs are more likely to trigger faults that remain undetected under typical, syntactically valid but semantically ordinary test cases produced by LLMs.

For instance, consider the classic buffer overflow scenario caused by using `scanf("%s", ...)` without proper bounds checking, as shown in Listing 3:

```
1 char buf[8];
2 scanf("%s", buf); // unsafe: no length limit
3 printf("Input: %s\n", buf);
```

Listing 3. Unsafe input handling vulnerable to non-printable or long inputs.

An LLM may generate a syntactically correct translation of the original COBOL code along with reasonable test inputs (e.g., "hello") that appear valid. Symbolic execution, however, can systematically produce malformed or non-printable inputs such as `"\x01\x02AAAAAAA"`, which can overflow the buffer or disrupt downstream functions. While non-printable characters alone may not immediately crash the program, they can be interpreted as control characters in `printf("Input: %s\n", buf)`; or, if a null byte (`\x00`) occurs early in the input, prematurely terminate the string, causing subsequent logic to make invalid assumptions.

From a coverage perspective, both the LLM- and symbolic execution-generated test suites may reach the same statements, giving the impression of equivalent coverage. Yet, only the symbolic execution inputs expose the underlying vulnerability, thereby revealing latent bugs that LLM-generated test cases may fail to detect.

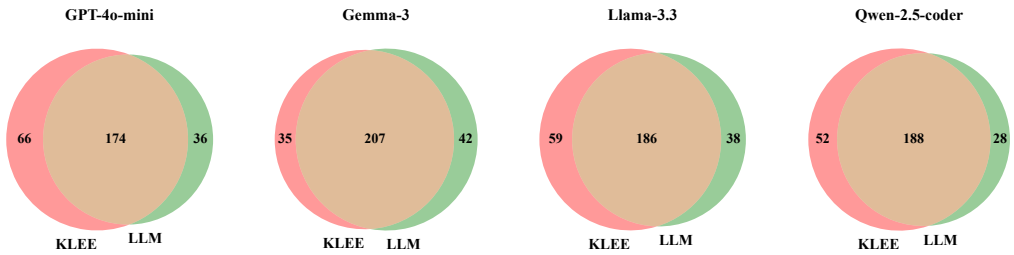


Fig. 3. Comparison of Bug-Triggering Effectiveness: Symbolic Execution vs. LLM-Generated Test Cases.

Furthermore, as illustrated in Figure 3, which shows the number of initially translated programs that failed testing using test cases generated by symbolic execution tools or LLMs, although many bugs can indeed be triggered by test cases from both sources, symbolic execution-based test cases expose a greater number of unique bugs overall. This advantage arises from their ability to

systematically generate corner cases that LLM-based test generation often overlooks, particularly for translated programs under minor input variations.

Summary 3: Although the test cases generated by symbolic execution tools and those produced by LLMs achieve comparable code coverage, the former are able to capture more corner cases. This makes them more effective in guiding LLMs to repair the translated code.

5.4 RQ4: How does the number of repair attempts affect the performance of SEDCoT?

Table 9. Impact of repair attempt numbers on the performance of different variants across LLMs (Higher scores denote better performance).

	Stage	Gemma-3	GPT-4o-mini	Llama-3.3	Qwen-2.5-coder
SEDCoT _{LLMTest}	CEF	0.315	0.414	0.384	0.349
	Repair-1	0.382 (↑ 21.3%)	0.472 (↑ 14.0%)	0.415 (↑ 8.1%)	0.400 (↑ 14.6%)
	Repair-2	0.385 (↑ 0.8%)	0.476 (↑ 0.8%)	0.436 (↑ 5.1%)	0.423 (↑ 5.8%)
	Repair-3	0.488 (↑ 26.8%)	0.547 (↑ 14.9%)	0.513 (↑ 17.7%)	0.503 (↑ 18.9%)
SEDCoT _{SymTest}	CEF	0.315	0.414	0.384	0.349
	Repair-1	0.483 (↑ 53.3%)	0.558 (↑ 34.8%)	0.467 (↑ 21.6%)	0.478 (↑ 37.0%)
	Repair-2	0.486 (↑ 0.6%)	0.565 (↑ 1.3%)	0.514 (↑ 10.1%)	0.499 (↑ 4.4%)
	Repair-3	0.601 (↑ 23.7%)	0.663 (↑ 17.3%)	0.594 (↑ 15.6%)	0.630 (↑ 26.3%)
SEDCoT _{w/oDelta}	CEF	0.315	0.414	0.384	0.349
	Repair-1	0.460 (↑ 46.0%)	0.590 (↑ 42.5%)	0.472 (↑ 22.9%)	0.479 (↑ 37.2%)
	Repair-2	0.471 (↑ 2.4%)	0.603 (↑ 2.2%)	0.510 (↑ 8.1%)	0.525 (↑ 9.6%)
	Repair-3	0.475 (↑ 0.8%)	0.586 (↓ 2.8%)	0.520 (↑ 2.0%)	0.568 (↑ 8.2%)
SEDCoT	CEF	0.315	0.414	0.384	0.349
	Repair-1	0.460 (↑ 46.0%)	0.590 (↑ 42.5%)	0.472 (↑ 22.9%)	0.479 (↑ 37.2%)
	Repair-2	0.471 (↑ 2.4%)	0.603 (↑ 2.2%)	0.510 (↑ 8.1%)	0.525 (↑ 9.6%)
	Repair-3	0.579 (↑ 22.9%)	0.718 (↑ 19.1%)	0.607 (↑ 19.0%)	0.668 (↑ 27.2%)

To investigate repair iterations, we evaluate translation accuracy across successive attempts, denoted in Table 9 as CEF (compilation error fixing via LLMs with up to two attempts) and Repair- N (the N -th repair round with test cases). Bracketed percentages indicate relative improvements over the preceding step. Comparing CEF and Repair-1 reveals that initial accuracy remains low after compilation fixes, whereas test-case feedback in the first attempt substantially boosts performance. However, this gain diminishes in Repair-2. By Repair-3, improvement for SEDCoT_{w/oDelta} stagnates, and for certain models like GPT-4o-mini, accuracy slightly degrades.

Conversely, delta debugging successfully sustains the LLM's repair capability for complex cases, as evidenced by comparing Repair-2 and Repair-3 across SEDCoT_{LLMTest}, SEDCoT_{SymTest}, and SEDCoT. Regardless of whether test suites are synthesized via symbolic execution, LLMs, or their combination, delta debugging consistently enhances repair effectiveness.

Listing 4 illustrates this by isolating a subtle input-related bug. The faulty translation uses `scanf("%s", ...)` to buffer input, followed by `sscanf(...)` to parse two integers. Because `scanf("%s")` terminates at the first whitespace, the buffer captures only a single token, causing `sscanf` to leave the second variable (`y`) uninitialized. While this defect remains masked under seemingly valid inputs like "10 2" due to residual memory contents, delta debugging systematically minimizes the failure-inducing input to "1". This failure forces a silent parsing error for `y`, localizing the root

cause to improper input handling rather than downstream logic. The robust version resolves this by utilizing `fgets` to capture full lines, initializing all variables, and explicitly verifying the parsed count.

```

1 /* Fragile: only reads a single word from input */
2 char buf[16]; long long x, y;
3 scanf("%15s", buf); // reads only up to first space
4 sscanf(buf, "%lld %lld", &x, &y); // can set x, but y remains
   uninitialized
5
6 /* Robust: reads full line and validates number of inputs */
7 char buf[16] = ""; long long x = 0, y = 0;
8 if (fgets(buf, sizeof(buf), stdin) &&
9     sscanf(buf, "%lld %lld", &x, &y) == 2) {
10     /* ok: both x and y successfully parsed */

```

Listing 4. Delta Debugging exposes uninitialized input.

Thus, delta debugging not only isolates faulty behaviors but also clarifies the nature of the failure, guiding LLMs toward structurally correct repairs. It complements test cases by converting them into minimal examples that sharpen fault localization and expose hidden assumptions in the code, revealing latent bugs that LLM-generated cases might miss.

Summary 4: Incorporating test cases into code repair improves performance, though the marginal benefit diminishes with more attempts, where excessive iterations can even degrade correctness. Integrating delta debugging successfully sustains this repair capability, especially for complex cases where test-feedback alone proves insufficient for LLMs.

5.5 RQ5: How readable is the code generated by SEDCoT compared to rule-based approaches?

Table 10. Comparison of Code Readability in SEDCoT Using Different LLMs and a Rule-Based Approach.

Dataset		Ground-Truth	Gemma-3	GPT-4o-mini	Llama-3.3	Qwen-2.5-coder	GnuCOBOL
Subset	Human	4.35	3.78	4.05	3.85	3.83	1.34
	LLM	3.40	4.20	4.20	4.00	4.30	1.70
Entire dataset	LLM	3.60	3.17	3.58	3.69	3.59	1.47

To evaluate readability, we conduct both human and automated experiments. For the human evaluation, we select 10 test cases where the code translated by all LLMs achieves a passing rate exceeding 97%. Twelve computer science students (2 undergraduates, 5 Master’s, 5 PhDs) rate the readability of the translated and ground-truth C programs on a 5-point Likert scale (1: extremely poor, 5: excellent), following established guidelines [6, 47, 55]. For the automated evaluation, we employ Grok Fast 1 with the same prompt guidelines to assess code readability. The complete evaluation guidelines are available in our replication package’s appendix.

Table 10 summarizes the results of both automated and human evaluations. In the human evaluation, the ground-truth code (original human-written code provided in the CodeNet dataset) achieves the highest readability score, closely followed by SEDCoT’s translations. Conversely, the rule-based approach scores significantly lower than both. For the automated LLM evaluation, despite minor scoring inconsistencies with human judgment (e.g., Qwen-2.5-coder receiving the highest

score), the overarching trend remains identical. Across the entire dataset, the LLM evaluation confirms that human-written and SEDCoT-translated code exhibit highly comparable readability, both substantially outperforming the rule-based baseline. These findings demonstrate that while rule-based translations suffer from poor readability, SEDCoT produces highly human-readable code, confirming its effectiveness.

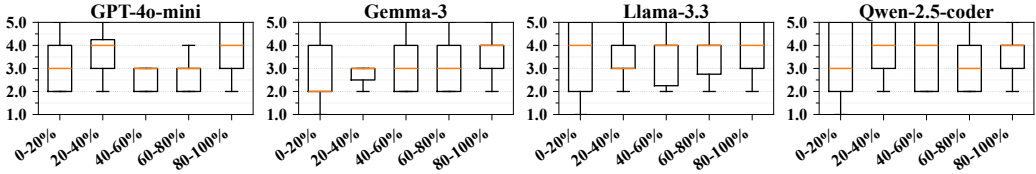


Fig. 4. Correlation between translated code correctness and readability (The X-axis denotes the pass rate of golden test suites, while the Y-axis indicates readability scores).

To explore the relationship between readability and functional correctness, we perform a stratified analysis of readability scores across 20-percentage-point intervals of test case passing rates. Intuitively, a positive correlation might be expected, where higher functional correctness corresponds to superior code readability. However, as illustrated in Figure 4, readability scores exhibit no apparent trend across varying levels of correctness. These results indicate that the readability of LLM-translated code is largely independent of its functional correctness.

Summary 5: SEDCoT translates COBOL into C with readability close to human-written code and significantly superior to rule-based approaches. Moreover, our evaluation demonstrates that the readability of LLM-translated code is largely independent of its functional correctness.

6 Discussion

In this section, we analyze the specific COBOL code structures—including fixed-width record layout, slicing, loop bounds, and output reconstruction—that may lead to translation failures.

6.1 Failed Translation Case of COBOL Code with Fixed-Width Record Layout

Listing 5 shows the COBOL code with a fixed-width record layout, and Listing 6 presents the corresponding translated C code. In this example, the COBOL code defines grouped input records with FILLER fields that encode fixed separator positions within the record layout. The ACCEPT INP1 statement reads data directly into this grouped structure. However, LLMs employ a simple char array of length three and read input via whitespace-separated scanning, which also differs from COBOL’s positional layout.

```

1 01 INP1 .
2   03 N PIC 9.
3   03 FILLER PIC X.
4   03 M PIC 9.
5   ...
6 ACCEPT INP1.
7 PERFORM VARYING I FROM 1 BY 1 UNTIL I > M
8   ACCEPT INP2
9 END-PERFORM.

```

Listing 5. COBOL code with fixed-width record layout

```

1 int main() {
2     char inp1[3];
3     ...
4     scanf("%1d %1d", &n, &m);
5
6     for (i = 1; i <= m; i++) {
7         scanf("%1d %1d", &s, &c);
8         ...
9     }
10 }

```

Listing 6. Translated C code with fixed-width record layout

6.2 Failed Translation Case of COBOL Code with Slicing, Loop Bounds, and Output Reconstruction

Listing 7 showcases several COBOL-specific semantic conventions, whose incorrect C translations are shown in Listing 8.

```

1 COMPUTE len = FUNCTION STORED-CHAR-LENGTH(S).
2 ...
3 PERFORM VARYING i FROM 1 BY -1 UNTIL i = 1
4 ...
5 IF S(1:len2) NOT = S(ini:len2) THEN ... END-IF
6 END-PERFORM.
7
8 MOVE i TO ZS.
9 PERFORM UNANS.
10 DISPLAY ans(1:FUNCTION STORED-CHAR-LENGTH(ans)).
11
12 UNANS SECTION.
13 UNSTRING ZS DELIMITED BY ALL SPACE INTO DUMMY ans END-UNSTRING.

```

Listing 7. COBOL code with slicing, loop bounds, and output reconstruction

First, the substring expression $S(ini:len2)$ relies on 1-based indexing, but is translated as $S + ini$ instead of the required $S + (ini - 1)$. Second, the loop `PERFORM VARYING i FROM 1 BY -1 UNTIL i = 1` is incorrectly mapped to `for (i = 1; i >= 1; i--)`. Because COBOL assumes TEST BEFORE semantics, the condition must be checked before execution, meaning the iteration for $i = 1$ should not run. Third, the translation collapses COBOL's complex field-level rendering and input/output

handling (ACCEPT, UNSTRING, STORED-CHAR-LENGTH) into ordinary C string operations (fgets, strlen, sscanf). Specifically, the original logic splits a field via UNSTRING ... DELIMITED BY ALL SPACE for output reconstruction, whereas the generated C code improperly formats and prints a null-terminated string directly.

```

1  int main() {
2      ...
3      for (i = 1; i >= 1; i--) {
4          ...
5          if (strcmp(S, S + ini, len2) != 0) { flg = 0; }
6          ...
7      }
8
9      snprintf(ZS, sizeof(ZS), "%d", i);
10     sscanf(ZS, "%s", ans);
11     printf("%s\n", ans);
12     return 0;
13 }
```

Listing 8. Translated C code with slicing, loop bounds, and output reconstruction

7 Related Work

7.1 Automated Code Translation

Research on code migration has progressed from manual rewriting and rule-based transcompilers to modern machine learning. Early methods leveraged foreign function interfaces like SWIG [3] or source-to-source transpilers like Emscripten [64]. Subsequent approaches applied statistical machine translation [26], repository mining [60], and unsupervised transformer models like TransCoder [52]. Recently, Pan et al. [43] introduced iterative repair via compiler and test feedback, which UniTrans [62] generalized across languages using LLM-generated tests. For COBOL translation, Gandhi et al. [17] proposed a three-phase refinement strategy. Other advancements explore reinforcement learning [23, 54], repository-level scaling [21], agentic workflows [33, 63], and semantic reasoning [13]. Unlike these code-level methods, alternative domain-level approaches generate functional descriptions [48] or leverage intermediate representations [28, 29] to guide the translation, whereas SEDCoT operates directly at the code level.

7.2 Using LLMs for Legacy Code

Legacy programming languages introduce unique challenges due to archaic syntax and hidden semantics. Diggs et al. [12] used LLMs to generate comments for MUMPS and ALC, while Ranasinghe et al. [49] and Chen et al. [9] achieved promising results in FORTRAN-to-C++ translation via fine-tuning. To manage complexity, Luo et al. [33] and Macedo et al. [34] proposed intermediate representation pipelines, whereas Lei et al. [30] used agentic methods for documentation. Additional studies focus on domain-specific rewrites with functional equivalence proofs [4] and architectural modernization [37] to ensure correctness at system scale. Such migration necessitates automated testing; search-based software testing (SBST) tools like Pynguin [32] implement MOSA [44] and DynaMOSA [45] strategies. Hybrid approaches combine SBST with LLMs, including CodaMOSA [31] and TELPA [61] for LLM-guided evolutionary testing, MuTAP for mutation testing [11], and CoverUp for iterative unit test generation [1]. Furthermore, symbolic execution tools like KLEE [7] and SymCC [46] systematically explore execution paths to uncover corner cases [2], with platforms like UTBot easing industrial application [18], extending to more languages

via fuzzing [58], or mitigating path explosion via machine learning [20]. Iterative LLM-based repair pipelines have also proven effective in resolving recurring error patterns [15, 43].

Building on these approaches, we combine symbolic execution with LLM-generated tests within a repair loop, apply delta debugging to minimize failing cases into counterexamples, and embed COBOL semantics in prompts for both functional correctness and readability. While readability is challenging to measure automatically due to metric limitations [14], empirical guidelines like reduced nesting [25] correlate with human perception and guide our evaluation. Additionally, SEDCoT's outputs are evaluated using LLMs for broader quality assessment, aligning with existing COBOL translation workflows [16] and general code evaluation practices [59, 66].

8 Threats to Validity

Although our empirical study demonstrates substantial improvements over state-of-the-art baselines, several threats may limit the generalizability of our results.

Accuracy gap with rule-based tools. While SEDCoT outperforms pure LLM baselines, its accuracy still lags behind mature rule-based COBOL translation tools that utilize engineered grammatical constraints. Consequently, our LLM-centric framework retains inherent limitations in fully matching the precision of industrial-grade solutions.

Evaluation data and scale. We evaluate SEDCoT on 319 function-level COBOL programs from IBM's CodeNet, which cover only a small fraction of real-world production code. Furthermore, our function-level approach does not handle complex repository-level file interactions, meaning performance on larger industrial scales remains to be verified.

Baselines and configurations. For reproducibility, we re-implemented UniTrans and HRJR using consistent prompts and deterministic decoding. However, alternative prompt templates, parameter settings (e.g., temperature sampling), or aggressive search strategies beyond our fixed repair iterations might yield different outcomes. Variations in LLM selection and deployment environments may also impact performance.

Potential data leakage. The evaluated COBOL programs might overlap with the proprietary training data of commercial LLMs. Due to the closed-source nature of these models and datasets, the exact extent of such performance inflation through memorization cannot be fully verified.

Target translation language selection. We select C as the target language because mature open-source COBOL compilers predominantly target C, and COBOL-to-C migration aligns with critical industrial requirements (e.g., AWS's legacy modernization plans [5]). Notably, while SEDCoT's underlying transcompiler can be interchanged to target other languages, performance may vary.

9 Conclusion

We introduced SEDCoT, a framework that translates COBOL into C by combining large language models (LLMs) with symbolic execution, automated test generation, and iterative repair. To handle COBOL's unique syntax and semantics, SEDCoT leverages delta debugging to simplify failing test cases, guiding the LLM toward accurate fixes. Our comprehensive evaluation demonstrates that SEDCoT consistently outperforms state-of-the-art baselines by at least 12%. These results showcase the effectiveness of integrating LLMs with symbolic and automated debugging techniques, providing a promising direction for legacy system modernization and future industrial applications.

Acknowledgements

This paper was written as part of the Elite Graduate Program in Software Engineering offered by the University of Augsburg, Technical University of Munich and Ludwig Maximilian University of Munich. ChatGPT has been used for linguistic optimizations and OpenAI's Codex has been used to draft code snippets.

Data Availability

The corresponding code and dataset have been made publicly available at <https://github.com/P-droi/issta2026-sedcot>.

References

- [1] Juan Altmayer Pizzorno and Emery D. Berger. 2025. CoverUp: Effective High Coverage Test Generation for Python. *Proceedings of the ACM on Software Engineering* 2, FSE (June 2025), 2897–2919. doi:10.1145/3729398
- [2] Joshua Bailey and Charles Nicholas. 2025. *Symbolic Execution in Practice: A Survey of Applications in Vulnerability, Malware, Firmware, and Protocol Analysis*. arXiv:2508.06643 [cs.CR] doi:10.48550/arXiv.2508.06643
- [3] David M. Beazley. 1996. SWIG: An Easy-to-Use Tool for Integrating Scripting Languages with C and C++. In *Proceedings of the Fourth Annual USENIX Tcl/Tk Workshop* (Monterey, CA, USA). USENIX Association, Berkeley, CA, USA, 129–139. https://www.usenix.org/publications/library/proceedings/tcl96/full_papers/beazley/index.html
- [4] Sahil Bhatia, Jie Qiu, Niranjan Hasabnis, Sanjit A. Seshia, and Alvin Cheung. 2024. Verified Code Transpilation with LLMs. In *Advances in Neural Information Processing Systems*, Amir Globerson, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub Tomczak, and Cheng Zhang (Eds.), Vol. 37. Curran Associates, Inc., Red Hook, NY, USA, 41394–41424. doi:10.52202/079017-1310
- [5] Veliswa Boya and Didier Durand. 2021. Serverless COBOL: Rejuvenating Legacy Code with Open Source Software. Accessed: 2026-05-21. <https://aws.amazon.com/blogs/opensource/serverless-cobol-rejuvenating-legacy-code-with-open-source-software/>
- [6] Raymond P. L. Buse and Westley Weimer. 2010. Learning a Metric for Code Readability. *IEEE Transactions on Software Engineering* 36, 4 (2010), 546–558. doi:10.1109/TSE.2009.70
- [7] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *8th USENIX Symposium on Operating Systems Design and Implementation (OSDI 08)* (San Diego, CA, USA). USENIX Association, Berkeley, CA, USA, 209–224. <https://www.usenix.org/conference/osdi-08/klee-unassisted-and-automatic-generation-high-coverage-tests-complex-systems>
- [8] David Cassel. 2017. *COBOL Is Everywhere. Who Will Maintain It?* Retrieved July 8, 2025 from <https://thenewstack.io/cobol-everywhere-will-maintain/>, archived at [<https://web.archive.org/web/20250612083600/https://thenewstack.io/cobol-everywhere-will-maintain/>]
- [9] Le Chen, Bin Lei, Dunzhi Zhou, Pei-Hung Lin, Chunhua Liao, Caiwen Ding, and Ali Jannesari. 2024. *Fortran2CPP: Automating Fortran-to-C++ Translation Using LLMs via Multi-Turn Dialogue and Dual-Agent Integration*. arXiv:2412.19770 [cs.LG] doi:10.48550/arXiv.2412.19770
- [10] Gary Cutler, Vincent Coen, Brian Tiffin, Bill Klein, László Erdős, Arnold Trembley, Edward Hart, Ron Norman, James K. Lowden, and Simon Sobisch. 2020. *GnuCOBOL Programmer's Guide*. Retrieved July 9, 2025 from <https://gnucobol.sourceforge.io/HTML/gnucobpg.html>, archived at [<https://web.archive.org/web/20250626160559/https://gnucobol.sourceforge.io/HTML/gnucobpg.html>]
- [11] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2024. Effective Test Generation Using Pre-Trained Large Language Models and Mutation Testing. *Information and Software Technology* 171 (July 2024), 107468. doi:10.1016/j.infsof.2024.107468
- [12] Colin Diggs, Michael Doyle, Amit Madan, Eric O. Scott, Emily Escamilla, Jacob Zimmer, Naveed Nekoo, Paul Ursino, Michael Bartholf, Zachary Robin, Anand Patel, Chris Glasz, William Macke, Paul Kirk, Jasper Phillips, Arun Sridharan, Doug Wendt, Scott Rosen, Nitin Naik, Justin F. Brunelle, and Samruddhi Thaker. 2025. Leveraging LLMs for Legacy Code Modernization: Evaluation of LLM-Generated Documentation. In *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*. IEEE, Piscataway, NJ, USA, 177–184. doi:10.1109/LLM4Code66737.2025.00027
- [13] Yangruibo Ding, Jinjun Peng, Marcus J. Min, Gail Kaiser, Junfeng Yang, and Baishakhi Ray. 2024. SemCoder: Training Code Language Models with Comprehensive Semantics Reasoning. In *Advances in Neural Information Processing Systems*, Amir Globerson, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub Tomczak, and Cheng Zhang (Eds.), Vol. 37. Curran Associates, Inc., Red Hook, NY, USA, 60275–60308. doi:10.52202/079017-1927
- [14] Sarah Fakhoury, Devjeet Roy, Adnan Hassan, and Vernera Arnaudova. 2019. Improving Source Code Readability: Theory and Practice. In *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*. IEEE, Piscataway, NJ, USA, 2–12. doi:10.1109/ICPC.2019.00014
- [15] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated Repair of Programs from Large Language Models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, Piscataway, NJ, USA, 1469–1481. doi:10.1109/ICSE48619.2023.00128
- [16] Shmulik Froimovich, Raviv Gal, Wesam Ibraheem, and Avi Ziv. 2025. *Quality Evaluation of COBOL-to-Java Code Transformation*. arXiv:2507.23356 [cs.SE] doi:10.48550/arXiv.2507.23356

- [17] Shubham Gandhi, Manasi Patwardhan, Jyotsana Khatri, Lovekesh Vig, and Raveendra Kumar Medicherla. 2024. Translation of Low-Resource COBOL to Logically Correct and Readable Java Leveraging High-Resource Java Refinement. In *Proceedings of the 1st International Workshop on Large Language Models for Code (LLM4Code '24)*. Association for Computing Machinery, New York, NY, USA, 46–53. doi:10.1145/3643795.3648388
- [18] Samat Gaynutdinov, Saveliy Grigoryev, Pavel Iatchenii, Elena Ilina, Dmitry Ivanov, Vladislav Kalugin, Aleksei Pleshakov, Pavel Ponomarev, Konstantin Rybkin, Svetlana Schmidt, Vadim Volodin, and Alexey Utkin. 2022. Presentation: UTBot Simplifies Auto Test Generation. Accessed: 2026-05-20. https://www.utbot.org/static/KLEE_workshop2022_abstract-9591232a9941df34577a134609dbbe29.pdf
- [19] Bernard Giroud, Brian Tiffin, Keisuke Nishida, Simon Sobisch, and Roger While. 2025. *GnuCOBOL—A Free COBOL Compiler*. Retrieved August 25, 2025 from <https://sourceforge.net/projects/gnucobol/>, archived at [<https://web.archive.org/web/20250825030006/https://sourceforge.net/projects/gnucobol/>]
- [20] Jingxuan He, Gishor Sivanrupan, Petar Tsankov, and Martin Vechev. 2021. Learning to Explore Paths for Symbolic Execution. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (Virtual Event, Republic of Korea) (CCS '21)*. Association for Computing Machinery, New York, NY, USA, 2526–2540. doi:10.1145/3460120.3484813
- [21] Ali Reza Ibrahimzada, Kaiyao Ke, Mrigank Pawagi, Muhammad Salman Abid, Rangeet Pan, Saurabh Sinha, and Reyhaneh Jabbarvand. 2025. AlphaTrans: A Neuro-Symbolic Compositional Approach for Repository-Level Code Translation and Validation. *Proceedings of the ACM on Software Engineering* 2, FSE (June 2025), 2454–2476. doi:10.1145/3729379
- [22] International Organization for Standardization. 2023. ISO/IEC 1989:2023—Information Technology—Programming Languages, Their Environments and System Software Interfaces—Programming Language COBOL. Accessed: 2026-05-21. <https://www.iso.org/standard/74527.html>
- [23] Prithwish Jana, Piyush Jha, Haoyang Ju, Gautham Kishore, Aryan Mahajan, and Vijay Ganesh. 2024. CoTran: An LLM-Based Code Translator Using Reinforcement Learning with Feedback from Compiler and Symbolic Execution. In *ECAI 2024 (Frontiers in Artificial Intelligence and Applications, Vol. 392)*. IOS Press, Amsterdam, The Netherlands, 4011–4018. doi:10.3233/FAIA240968
- [24] Mingsheng Jiao, Tingrui Yu, Xuan Li, Guanjie Qiu, Xiaodong Gu, and Beijun Shen. 2023. On the Evaluation of Neural Code Translation: Taxonomy and Benchmark. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Piscataway, NJ, USA, 1529–1541. doi:10.1109/ASE56229.2023.00114
- [25] John Johnson, Sergio Lobo, Nishitha Yedla, Jairo Aponte, and Bonita Sharif. 2019. An Empirical Study Assessing Source Code Readability in Comprehension. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, Piscataway, NJ, USA, 513–523. doi:10.1109/ICSME.2019.00085
- [26] Svetoslav Karaivanov, Veselin Raychev, and Martin Vechev. 2014. Phrase-Based Statistical Translation of Programming Languages. In *Proceedings of the 2014 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software (Portland, Oregon, USA) (Onward! 2014)*. Association for Computing Machinery, New York, NY, USA, 173–184. doi:10.1145/2661136.2661148
- [27] Atul Kumar, Diptikalyan Saha, Toshiaki Yasue, Kohichi Ono, Saravanan Krishnan, Sandeep Hans, Fumiko Satoh, Gerald Mitchell, and Sachin Kumar. 2024. Automated Validation of COBOL-to-Java Transformation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*. Association for Computing Machinery, New York, NY, USA, 2415–2418. doi:10.1145/3691620.3695365
- [28] K. Lano, P. T. Breuer, and H. Haughton. 1993. Reverse-Engineering COBOL via Formal Methods. *Journal of Software Maintenance: Research and Practice* 5, 1 (1993), 13–35. doi:10.1002/smr.4360050103
- [29] Kevin Lano and Hanan Siala. 2024. Using model-driven engineering to automate software language translation. *Automated Software Engineering* 31, 1, Article 20 (Feb. 2024), 59 pages. doi:10.1007/s10515-024-00419-y
- [30] Fangjian Lei, Jiawen Liu, Shayan Noei, Ying Zou, Derek Truong, and William Alexander. 2025. *Enhancing COBOL Code Explanations: A Multi-Agents Approach Using Large Language Models*. arXiv:2507.02182 [cs.SE] doi:10.48550/arXiv.2507.02182
- [31] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-Trained Large Language Models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, Piscataway, NJ, USA, 919–931. doi:10.1109/ICSE48619.2023.00085
- [32] Stephan Lukasczyk and Gordon Fraser. 2022. Pynguin: Automated Unit Test Generation for Python. In *2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, Piscataway, NJ, USA, 168–172. doi:10.1109/ICSE-Companion55297.2022.9793730
- [33] Wenqiang Luo, Jacky Wai Keung, Boyang Yang, Jacques Klein, Tegawende F. Bissyande, Haoye Tian, and Bach Le. 2025. *Unlocking LLM Repair Capabilities Through Cross-Language Translation and Multi-Agent Refinement*. arXiv:2503.22512 [cs.SE] doi:10.48550/arXiv.2503.22512

- [34] Marcos Macedo, Yuan Tian, Pengyu Nie, Filipe R. Cogo, and Bram Adams. 2024. *InterTrans: Leveraging Transitive Intermediate Translations to Enhance LLM-Based Code Translation*. arXiv:2411.01063 [cs.SE] doi:10.48550/arXiv.2411.01063
- [35] Carmelo Montanez-Rivera. 1999. *COBOL Test Suites*. Retrieved August 25, 2025 from https://www.itl.nist.gov/div897/ctg/cobol_form.htm, archived at [https://web.archive.org/web/20230917224831/https://www.itl.nist.gov/div897/ctg/cobol_form.htm]
- [36] Keisuke Nishida, Roger While, and Simon Sobisch. 2023. *GnuCOBOL—GNU Project*. Retrieved July 9, 2025 from <https://gnucobol.sourceforge.io/>, archived at [<https://web.archive.org/web/20250703071501/https://gnucobol.sourceforge.io/>]
- [37] Vikram Nitin. 2024. Using AI to Automate the Modernization of Legacy Software Applications. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 2514–2517. doi:10.1145/3691620.3695610
- [38] OpenAI. 2025. *GPT-4o mini*. Retrieved August 5, 2025 from <https://platform.openai.com/docs/models/gpt-4o-mini>, archived at [<https://web.archive.org/web/20250726062749/https://platform.openai.com/docs/models/gpt-4o-mini>]
- [39] OpenRouter. 2026. OpenRouter. Accessed: 2026-05-21. <https://openrouter.ai/>
- [40] OpenRouter, Inc. 2024. *Meta: Llama 3.3 70B Instruct*. Retrieved August 5, 2025 from <https://openrouter.ai/meta-llama/llama-3.3-70b-instruct>, archived at [<https://web.archive.org/web/20250715004723/https://openrouter.ai/meta-llama/llama-3.3-70b-instruct>]
- [41] OpenRouter, Inc. 2024. *Qwen2.5 Coder 32B Instruct*. Retrieved August 5, 2025 from <https://openrouter.ai/qwen/qwen-2.5-coder-32b-instruct>, archived at [<https://web.archive.org/web/20250709025049/https://openrouter.ai/qwen/qwen-2.5-coder-32b-instruct>]
- [42] OpenRouter, Inc. 2025. *Google: Gemma 3 27B*. Retrieved August 5, 2025 from <https://openrouter.ai/google/gemma-3-27b-it>, archived at [<https://web.archive.org/web/20250719132359/https://openrouter.ai/google/gemma-3-27b-it>]
- [43] Rangeet Pan, Ali Reza Ibrahimzade, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in Translation: A Study of Bugs Introduced by Large Language Models while Translating Code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, 1–13. doi:10.1145/3597503.3639226
- [44] Annibale Panichella, Fitsum Meshesha Kifetew, and Paolo Tonella. 2015. Reformulating Branch Coverage as a Many-Objective Optimization Problem. In *2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, Piscataway, NJ, USA, 1–10. doi:10.1109/ICST.2015.7102604
- [45] Annibale Panichella, Fitsum Meshesha Kifetew, and Paolo Tonella. 2018. Automated Test Case Generation as a Many-Objective Optimisation Problem with Dynamic Selection of the Targets. *IEEE Transactions on Software Engineering* 44, 2 (Feb. 2018), 122–158. doi:10.1109/TSE.2017.2663435
- [46] Sebastian Poepplau and Aurélien Francillon. 2020. Symbolic Execution with SymCC: Don't Interpret, Compile!. In *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Berkeley, CA, USA, 181–198. <https://www.usenix.org/conference/usenixsecurity20/presentation/poepplau>
- [47] Daryl Posnett, Abram Hindle, and Premkumar T. Devanbu. 2011. A Simpler Model of Software Readability. In *Proceedings of the 8th Working Conference on Mining Software Repositories* (Waikiki, Honolulu, HI, USA), Arie van Deursen, Tao Xie, and Thomas Zimmermann (Eds.). Association for Computing Machinery, New York, NY, USA, 73–82. doi:10.1145/1985441.1985454
- [48] Asha Rajbhoj, Akanksha Somase, Tanay Sant, Ajim Pathan, Purvesh Doud, and Vinay Kulkarni. 2025. Leveraging LLM for Software Modernization: COBOL Functionality Extraction Case Study. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*. IEEE, Piscataway, NJ, USA, 14–21. doi:10.1109/ASEW67777.2025.00012
- [49] Nishath Rajiv Ranasinghe, Shawn M. Jones, Michal Kucer, Ayan Biswas, Daniel O'Malley, Alexander Buschmann Most, Selma Liliane Wanna, and Ajay Sreekumar. 2025. *LLM-Assisted Translation of Legacy FORTRAN Codes to C++: A Cross-Platform Study*. arXiv:2504.15424 [cs.SE] doi:10.48550/arXiv.2504.15424
- [50] Matthew Renze. 2024. The Effect of Sampling Temperature on Problem Solving in Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 7346–7356. doi:10.18653/v1/2024.findings-emnlp.432
- [51] Reuters Graphics. 2017. *COBOL Blues*. Retrieved August 5, 2025 from <https://www.reuters.com/graphics/USA-BANKS-COBOL/010040KH18J/>, archived at [<https://web.archive.org/web/20250726190454/https://www.reuters.com/graphics/USA-BANKS-COBOL/010040KH18J/>]
- [52] Baptiste Roziere, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. 2020. Unsupervised Translation of Programming Languages. In *Advances in Neural Information Processing Systems*, Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.), Vol. 33. Curran Associates, Inc., Red Hook, NY, USA, 20601–20611. <https://proceedings.neurips.cc/paper/2020/hash/ed23bfb18c2cd35f8c7f8de44f85c08d-Abstract.html>

- [53] Yutaro Sakamoto. 2025. *opensource COBOL 4j*. <https://github.com/opensourcecobol/opensourcecobol4j>
- [54] Marina Sakharova, Abhinav Anand, and Mira Mezini. 2025. Integrating Symbolic Execution into the Fine-Tuning of Code-Generating LLMs. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 4: Student Research Workshop)*, Abteem Ebrahimi, Samar Haider, Emmy Liu, Sammar Haider, Maria Leonor Pacheco, and Shira Wein (Eds.). Association for Computational Linguistics, Albuquerque, USA, 271–278. doi:10.18653/v1/2025.naacl-srw.27
- [55] Agnia Sergeyuk, Olga Lvova, Sergey Titov, Anastasiia Serova, Farid Bagirov, and Timofey Bryksin. 2024. *Assessing Consensus of Developers' Views on Code Readability*. arXiv:2407.03790 [cs.SE] doi:10.48550/arXiv.2407.03790
- [56] Kathrin Stoll. 2020. *USA suchen dringend Programmierer für uralte Behördensysteme*. Retrieved August 5, 2025 from <https://www.welt.de/wirtschaft/webwelt/article207536129/Cobol-USA-suchen-wegen-Coronakrise-Programmierer-fuer-alte-Systeme.html>, archived at [<https://web.archive.org/web/20240702212854/https://www.welt.de/wirtschaft/webwelt/article207536129/Cobol-USA-suchen-wegen-Coronakrise-Programmierer-fuer-alte-Systeme.html>]
- [57] Ekaterina Tochilina, Vyacheslav Tamarin, Dmitry Mordvinov, Valentyn Sobol, Sergey Pospelov, Alexey Menshutin, Yury Kamenev, and Dmitry Ivanov. 2024. UTBot Python at the SBFT Tool Competition 2024. In *Proceedings of the 17th ACM/IEEE International Workshop on Search-Based and Fuzz Testing (Lisbon, Portugal) (SBFT '24)*. Association for Computing Machinery, New York, NY, USA, 41–42. doi:10.1145/3643659.3643934
- [58] Ekaterina Tochilina, Vyacheslav Tamarin, Dmitry Mordvinov, Valentyn Sobol, Sergey Pospelov, Alexey Menshutin, Yury Kamenev, and Dmitry Ivanov. 2024. UTBot Python at the SBFT Tool Competition 2024. In *Proceedings of the 17th ACM/IEEE International Workshop on Search-Based and Fuzz Testing (Lisbon, Portugal) (SBFT '24)*. Association for Computing Machinery, New York, NY, USA, 41–42. doi:10.1145/3643659.3643934
- [59] Weixi Tong and Tianyi Zhang. 2024. CodeJudge: Evaluating Code Generation with Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 20032–20051. doi:10.18653/v1/2024.emnlp-main.1118
- [60] Fengcai Wen, Emad Aghajani, Csaba Nagy, Michele Lanza, and Gabriele Bavota. 2021. Siri, Write the Next Method. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, Piscataway, NJ, USA, 138–149. doi:10.1109/ICSE43902.2021.00025
- [61] Chen Yang, Junjie Chen, Bin Lin, Ziqi Wang, and Jianyi Zhou. 2026. Advancing Code Coverage: Incorporating Program Analysis with Large Language Models. *ACM Transactions on Software Engineering and Methodology* 35, 5 (April 2026), 1–31. doi:10.1145/3748505
- [62] Zhen Yang, Fang Liu, Zhongxing Yu, Jacky Wai Keung, Jia Li, Shuo Liu, Yifan Hong, Xiaoxue Ma, Zhi Jin, and Ge Li. 2024. Exploring and Unleashing the Power of Large Language Models in Automated Code Translation. *Proceedings of the ACM on Software Engineering* 1, FSE, Article 71 (July 2024), 24 pages. doi:10.1145/3660778
- [63] Zhiqiang Yuan, Weitong Chen, Hanlin Wang, Xin Peng, Zhenpeng Chen, and Yiling Lou. 2026. TransAgent: Enhancing LLM-Based Code Translation via Fine-Grained Execution Alignment. *Proceedings of the ACM on Software Engineering* 3, FSE (June 2026), 1582–1604. doi:10.1145/3797099
- [64] Alon Zakai. 2011. Emscripten: An LLVM-to-JavaScript Compiler. In *Proceedings of the ACM International Conference Companion on Object Oriented Programming Systems Languages and Applications Companion (Portland, Oregon, USA) (OOPSLA '11)*. Association for Computing Machinery, New York, NY, USA, 301–312. doi:10.1145/2048147.2048224
- [65] Andreas Zeller. 1999. Yesterday, My Program Worked. Today, It Does Not. Why? *ACM SIGSOFT Software Engineering Notes* 24, 6 (Oct. 1999), 253–267. doi:10.1145/318774.318946
- [66] Terry Yue Zhuo. 2024. ICE-Score: Instructing Large Language Models to Evaluate Code. In *Findings of the Association for Computational Linguistics: EACL 2024*, Yvette Graham and Matthew Purver (Eds.). Association for Computational Linguistics, St. Julian's, Malta, 2232–2242. doi:10.18653/v1/2024.findings-eacl.148

Received 2026-01-29; accepted 2026-06-25